ClassLink: A BLE-Based Classroom Attendance System

Project Proposal for UCS503P
Submitted to: Dr. Raghav B. Venkataramaiyer
Thapar Institute of Engineering and Technology

Vemula Manvi Smaran, CSED* Vrishank Nehru, CSED† Mahi Singh, CSED‡
Khagendra Saini, CSED§

October 15, 2026

1 Higher-order goal

. . . secondary framing

This project aims to reduce administrative overhead and time lost to manual attendance-taking in classrooms, freeing instructional time and producing a reliable, auditable attendance record. The broader framing is *efficient, low-friction classroom administration*; the immediate engineering objective is to translate that into a measurable, deployable mobile/backend system.

2 Time-to-value

. . . primary heuristic

- We deliver meaningful increments quickly: an offline proof-of-concept validating the core Bluetooth Low Energy (BLE) proximity mechanism was built first, before any backend or auth work.
- We prioritize fast feedback over perfection by working in vertical slices (auth → classrooms → device binding → BLE → attendance transaction → teacher operations), keeping the app runnable after every phase.
- Success in the first iteration is defined narrowly: prove that a teacher phone can advertise a session over BLE and a student phone can detect it by proximity and check in, with a working CI pipeline running analyzers and tests on every change.

3 Problem Statement

Manual, roll-call-based classroom attendance in a college setting is slow, easy to proxy, and produces attendance data that is hard to audit or export. Common pain points include:

- **Time cost:** calling out or circulating a sheet consumes several minutes of every lecture.

*Roll No: 1024160111
†Roll No: 1024160109
‡Roll No: 1024160102
§Roll No: 1024160123

- **Proxy attendance:** a physical sheet or verbal roll call cannot verify that the responding student is actually present.
- **No live visibility:** the teacher has no real-time roster of who has checked in during the session.
- **Poor exportability:** paper or ad-hoc digital records are difficult to aggregate into per-course, per-student attendance percentages for reporting.

Why this matters now (contextual relevance): the problem is directly experienced by our own project team as students, in our own classrooms, making it both immediately relevant and immediately testable with real hardware and real class sessions.

4 Proposed Solution

. . . Software Proposition

4.1 Overview

Build **ClassLink**, an Android-first Flutter mobile application backed by a FastAPI service and a Supabase/PostgreSQL database, with the following components:

- **Teacher app role:** create classrooms, share a join code, start/stop a BLE attendance session, and view a live roster.
- **Student app role:** join a classroom once, then scan for an active session's BLE beacon and submit a signed attendance claim.
- **Server-authoritative backend:** validates enrollment, session state, token freshness, device binding, duplicate claims, and proximity evidence before accepting an attendance record.
- **Audit trail:** every security-sensitive action (device binding, manual correction, export) is logged as an audit event.

4.2 Core workflow

1. Teacher starts an attendance session; the phone begins advertising a short-lived, rotating BLE token identifying the session.
2. Enrolled student phones nearby scan for the token, collect several RSSI (signal strength) samples, and apply a configured proximity rule.
3. On a locally authenticated device, the student's phone signs and submits an attendance claim to the backend.
4. The backend validates the claim end-to-end and returns a server-authoritative result; the teacher's live roster updates accordingly.

4.3 Operational constraints for an educational, single-classroom setting

- Target Android phones as the primary, tested platform; keep abstractions portable without claiming untested iOS support.
- No Bluetooth mesh or student-to-student relay in the first release — strictly teacher-advertises, student-scans.
- Keep the BLE advertisement payload small (at or below 20 bytes) and free of any student/teacher identifiers.

5 Solution Approach *... Engineering focus*

This is an engineering course project, so we focus on system correctness, security boundaries, and reproducible evaluation rather than novel algorithm research.

5.1 Proximity assistance *... non-research*

Proximity decisions are heuristic, not exact positioning:

- Treat RSSI as noisy evidence, never as proof of exact distance.
- Use the median of multiple observations within a sampling window rather than deciding on a single packet.
- Store the RSSI threshold and sample policy per session so experiments remain reproducible; do not present preset defaults as scientific findings.

5.2 Mobile + backend architecture *... BLE-based solution*

A three-tier architecture:

- **Mobile client (Flutter/Riverpod):** teacher advertiser/student scanner roles, local biometric/device authentication, offline-safe state handling.
- **Backend API (FastAPI):** authentication verification, session/token lifecycle, claim validation pipeline, roster and export endpoints.
- **Database (Supabase/PostgreSQL):** classrooms, enrollments, sessions, tokens, attendance records, and audit events, protected by row-level security policies.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Run `flutter analyze`, `dart format`, and Flutter unit/widget tests on every change.
- Run Ruff, MyPy, and Pytest for the backend, plus a SQL migration smoke test where feasible.
- Check that no `.env` secret files are committed.

6 Evaluation Criterion *... measurable and attributable*

We define success using measurable metrics tied to the core workflow, gathered from executed tests on real Android hardware — never pre-filled or assumed values.

6.1 Primary evaluation metric

Detection success rate: the fraction of labelled-position scan attempts (front, middle, back, door, corridor, adjacent room) that correctly accept or reject attendance according to the configured proximity policy.

6.2 Secondary evaluation metrics

- **Discovery latency:** time from teacher advertising start to first valid student packet.
- **Attendance completion latency:** time from scan start to server-authoritative result.
- **False accept / false reject rate:** incorrect accept/reject decisions at out-of-room and in-room test positions, respectively.
- **API latency:** p50/p95 latency for the challenge and claim endpoints.
- **Duplicate and expiry enforcement:** correct rejection rate for duplicate claims and expired tokens.

6.3 Pilot validation plan *... high-level*

- Run scenario tests with one teacher phone (advertiser) and at least two student phones (scanners) of different models, in an actual classroom.
- Record raw, timestamped CSV measurements for every scenario under `docs/evaluation/`, including phone model, Android version, room label, and position label, before deriving any summary tables or figures.

7 Scalability *... with foresight*

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** the system should support growth in number of classrooms, sessions, and concurrent scanning students by:
   - keeping BLE payloads compact and session-scoped,
   - indexing session/token lookups by alias and status,
   - making attendance writes idempotent under retry.
2. **Operationally deployable (practically):** the system should run on common infrastructure:
   - a managed PostgreSQL instance (Supabase),
   - a standard FastAPI/Uvicorn deployment,
   - a CI pipeline gating merges on analyzer, lint, and test results.

8 Engine availability heuristic

A mature set of "engines" (frameworks/services) is available to implement this as a mobile + backend project:

- Flutter/Dart for a single Android-first mobile codebase.
- A maintained BLE package supporting both central (scan) and peripheral (advertise) roles.
- FastAPI for a typed, testable REST backend.
- Supabase for managed PostgreSQL, auth, and row-level security.
- Pytest/Flutter test tooling for unit, widget, and integration tests.

We use these mature components to focus on workflow correctness, security boundaries, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables *... iteration-friendly*

9.1 Initial deliverable *... first iteration*

- Offline BLE proof-of-concept: teacher advertise, student scan and check-in by proximity, CSV export (**complete**).
- Supabase schema, authentication, roles, and RLS isolation tests.
- Classroom creation, join-code enrollment, teacher/student UI.
- CI: Flutter analyzer/tests and backend Ruff/MyPy/Pytest.

9.2 Subsequent deliverables

- Device-key binding (Android Keystore) and signed attendance claims.
- Rotating-token BLE sessions, server-authoritative claim validation, and live teacher roster.
- Manual correction workflow with mandatory reason and audit trail.
- CSV export and real-device evaluation dataset under `docs/evaluation/`.

10 Risks and mitigations

- **BLE peripheral-mode reliability on Android:** test only on physical devices; emulators do not reliably support GATT-server/ peripheral advertising.
- **Proxy/spoofing risk:** bind attendance claims to a device-generated signing key and require local biometric authentication before signing, rather than trusting a bare BLE detection.
- **Evaluation measurement ambiguity:** fix the metric definitions and CSV schema (Section 7) before running any device experiments.
- **Secret leakage:** keep the session-token HMAC secret and Supabase service-role key server-side only; commit only `.env.example` files.

11 Summary

This project proposes ClassLink, a BLE-based classroom attendance system that replaces manual roll call with a device-bound, server-authoritative check-in flow. It meets the course criteria by emphasizing time-to-value through an already-working offline proof-of-concept, implementing CI to support frequent safe releases, and using measurable, real-device evaluation criteria rather than assumed results. It remains focused on engineering correctness and security boundaries rather than research- driven novelty.